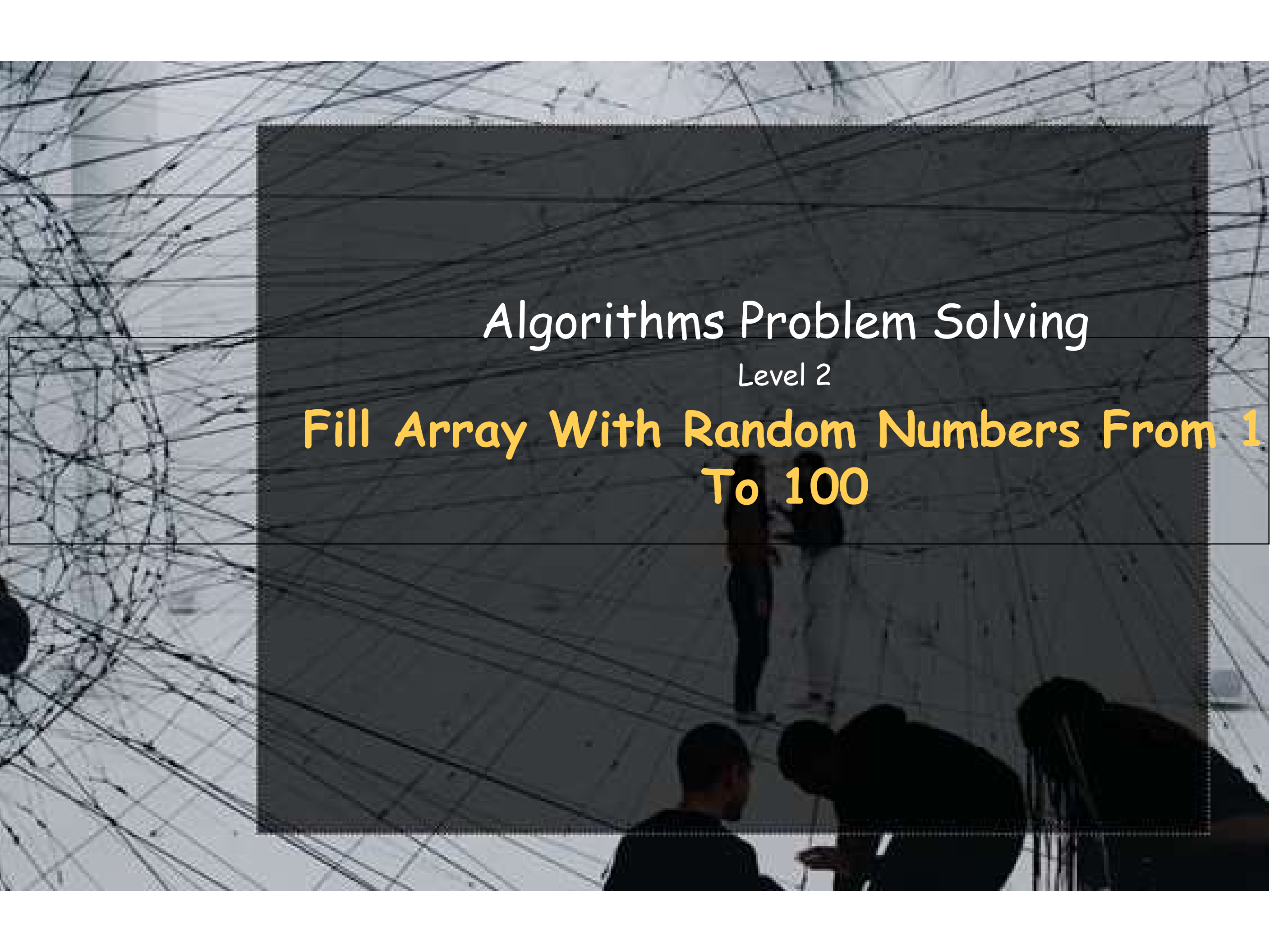


Algorithms Problem Solving

Level 2

Fill Array With Random
Numbers From 1 To 100

The background of the slide is a grayscale photograph of a basketball court. In the foreground, the silhouettes of several players are visible, some standing and some in motion. The court floor has a visible wood grain pattern. A large, dark, semi-transparent rectangular box is centered over the image, serving as a backdrop for the text.

Algorithms Problem Solving

Level 2

**Fill Array With Random Numbers From 1
To 100**

Definition

- ◆ A program that fills an array with a user-specified number of random integers from `1` to `100`.
- ◆ The array has a maximum capacity of `100` elements.
- ◆ After filling the array, the program prints all its elements.

Explanation

- ◆ ``FillArrayWithRandomNumber()`` asks the user how many elements should be stored in the array.
- ◆ It uses a ``for`` loop to visit each array position.
- ◆ ``RandomNumber(1, 100)`` generates a random number between ``1`` and ``100``.
- ◆ The generated number is stored in ``arr[i]``.
- ◆ ``PrintArrayElements()`` loops through the filled part of the array and prints every element.
- ◆ ``arrayLength`` is passed by reference using ``int&`` so the function can store the number of elements entered by the user and return it to ``main()``.

Steps in main()

1. Create an array with a maximum size of `100`.
2. Ask the user for the number of elements.
3. Store the number in `arrLength`.
4. Start a loop from `0` to `arrLength - 1`.
5. Generate a random number from `1` to `100`.
6. Store the random number in `arr[i]`.
7. Print all elements from index `0` to `arrLength - 1`.
8. Display the generated numbers.

💡 Example

◆ Input:

Enter number of elements: 10

◆ The program may generate:

42 68 35 1 70 25 79 59 63 65

◆ Another execution can produce completely different numbers.

➡ Result

◆ The output may be:

Array Elements: 42 68 35 1 70 25 79 59 63 65

◆ Every generated element is between `1` and `100`.

Note

- ◆ `arr[i] = RandomNumber(1, 100);` generates and stores a random number directly in the array.
- ◆ `int& arrLength` means `arrLength` is passed by reference, so changes made inside `FillArrayWithRandomNumber()` affect the variable `arrayLength` in `main()`.
- ◆ Only the first `arrayLength` positions of the array are used.
- ◆ The function `ReadArrayElements()` exists in the code but is not used because the program generates the values randomly instead of reading them from the user.

⊘ Warning

- ◆ `FillArrayWithRandomNumber()` does not validate that `arrLength` is between `1` and `100`.
- ◆ If the user enters a value greater than `100`, the program will write outside the array's valid range.
- ◆ The program does not call `srand((unsigned)time(NULL))`, so the random sequence may be the same each time the program runs.
- ◆ `<cmath>` and `<corecrt_math_defines.h>` are not needed for this program.

Comparison

- ▣ `ReadArrayElements()` → gets each array value from the user.
- ▣ `FillArrayWithRandomNumber()` → generates each array value randomly.
- ▣ `RandomNumber(1, 100)` → generates one random number between `1` and `100`.
- ▣ `PrintArrayElements()` → prints the values already stored in the array.

✦ Summary

- ✓ The program fills an array with random numbers.
- ✓ Each number is between `1` and `100`.
- ✓ The array has a maximum capacity of `100` elements.
- ✓ `FillArrayWithRandomNumber()` fills the array.
- ✓ `RandomNumber()` generates each random value.
- ✓ `int& arrLength` allows the function to return the chosen array length.
- ✓ `PrintArrayElements()` prints the generated elements.

Thank you 😊